Query Optimization: Choice of Evaluation Plans

1 Introduction

An **evaluation plan** defines exactly what algorithm should be used for each operation and how their execution should be coordinated. A cost-based optimizer explores the space of all possible evaluation plans equivalent to a given query and selects the one with the lowest estimated cost.

Optimization Heuristics

Exploring all plans can be prohibitively expensive for complex queries. Optimizers use heuristics and dynamic programming to reduce optimization time while aiming for an near-optimal result.

2 Cost-Based Join-Order Selection

Join order is often the most significant factor in query cost. For a join of n relations $r_1 \bowtie r_2 \bowtie \ldots \bowtie r_n$, the number of possible join orders grows exponentially.

2.1 Combinatorial Complexity

The total number of different join orderings for n relations is calculated as:

$$\mathcal{N} = \frac{(2(n-1))!}{(n-1)!}$$

Relations (n)	Possible Join Orders	Dynamic Programming Complexity
3	12	Small
5	1,680	Manageable
7	665,280	$O(3^n)$
10	$>$ 17.6 Billion	$\approx 59,049$

Table 1: Exponential growth of query search space vs. DP optimization.

3 Cost of Optimization

Cost-based optimization is expensive but essential for queries on large datasets. Dynamic programming reduces the search space significantly:

- **Bushy Trees:** Time complexity is $O(3^n)$ and space complexity is $O(2^n)$.
- **Left-Deep Trees:** Time complexity is reduced to $O(n2^n)$, while space remains $O(2^n)$.

3.1 Optimizing for Left-Deep Join Trees

Many optimizers restrict search to left-deep trees to reduce complexity and enable pipelining.

Optimization via Subsets

To find the best join tree for a set S of n relations:

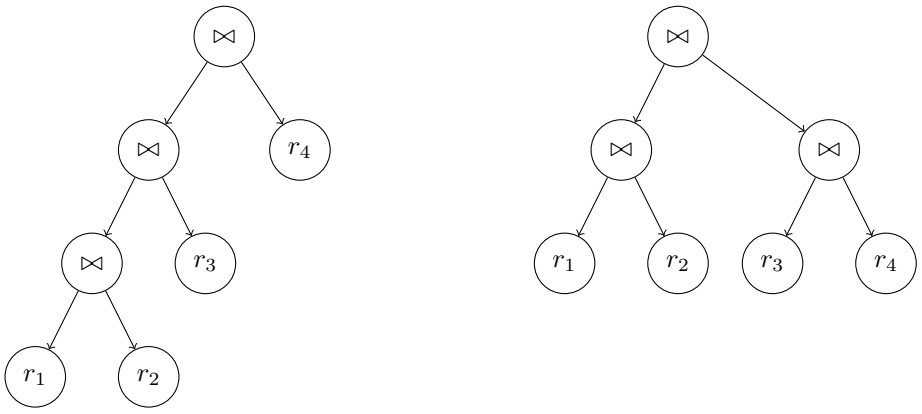
1. **Partitioning**: Consider all possible plans of the form $S_1 \bowtie (S - S_1)$ where S_1 is any non-empty subset of S.
2. **Recursion**: Join costs for subsets are computed recursively.
3. **Selection**: Choose the cheapest of the $2^n - 2$ alternatives.
4. **Base Case**: Single relation access plan (using best indices/selections).
5. **Reuse**: Store the plan for each subset and reuse it when needed.

FindBestPlan(S) - Dynamic Programming

```
procedure FindBestPlan(S)
    // Return cached result if already computed
    if bestPlan[S].cost != infinity then
        return bestPlan[S]

    // Base case: single relation
    if |S| = 1 then
        bestPlan[S] <- BestAccessMethod(S)
        return bestPlan[S]

    // Initialize cost to infinity
    bestPlan[S].cost <- infinity

    // Try all valid partitions of S
    // To restrict to Left-Deep: Use 'for each relation r in S'
       and 'S1 = S - r'
    for each non-empty subset S1 of S where S1 != S do
        S2 <- S - S1

        P1 <- FindBestPlan(S1)
        P2 <- FindBestPlan(S2)

        // Try all join algorithms
        for each join algorithm A in {HashJoin, MergeJoin,
           NestedLoop} do
            cost <- P1.cost + P2.cost + Cost(A)

            if cost < bestPlan[S].cost then
                bestPlan[S].plan <- Join(P1, P2, A)
                bestPlan[S].cost <- cost

    return bestPlan[S]
```

4 Join Tree Structures

Optimizers often restrict the *shape* of the join tree to simplify the search space.



Left-Deep Tree **Bushy Tree/Non-Left-deep Join Tree**

Figure 1: Different join tree topologies considered during optimization.

5 Interesting Sort Orders

An **interesting sort order** is a specific sequence of tuples that can make subsequent operations cheaper.

Synergy of Merge-Join

Consider $(r_1 \bowtie r_2) \bowtie r_3$ with common attribute A.

- A **Hash Join** for $(r_1 \bowtie r_2)$ might be cheaper initially but produces unsorted results.
- A **Merge Join** might be slightly more expensive for the first join but generates results sorted on A.
- This sorted result allows a very cheap Merge Join with r_3, reducing the *total* cost of the entire query.

To capture this, the optimizer must find the best plan for each subset **for each interesting sort order** $[S, o]$.

6 Custom Optimization Strategies

6.1 Optimization with Equivalence Rules

Physical equivalence rules convert logical query plans into physical plans. Modern systems (like SQL Server) use:

- **Space-efficient representations** to avoid subexpression redundancy.
- **Memoization** to store and reuse best sub-plans.
- **Cost-based pruning** to discard expensive paths early.

6.2 Heuristic Optimization

Heuristics transform the query tree using rules that typically improve performance without exhaustive search:

- **Early Selection/Projection:** Reduces the number of tuples and attributes as early as possible.
- **Restrictive First:** Perform joins with the smallest result sizes first.

7 Structure of Query Optimizers

Real-world optimizers often combine several approaches:

- **Pipelining Preference:** Many focus on left-deep trees to facilitate pipeline execution.
- **Block-Structure Orientation:** Rewrite nested block structures and perform join-order optimization for each block separately.
- **Budgeting:** Setting an **optimization cost budget** ensures that the time spent optimizing doesn't exceed the potential gain in execution time.
- **Plan Caching:** Reusing previously computed plans for resubmitted queries, even with different constants.

Optimization Overhead

While cost-based optimization is expensive, it is highly worthwhile for complex queries on large datasets. For very cheap queries, simple heuristics are often sufficient.